

```
$ ros2 service list
/clear
/kill
/reset
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/describe_parameters
/turtlesim/get_parameter_types
/turtlesim/get_parameters
/turtlesim/list_parameters
/turtlesim/set_parameters
/turtlesim/set_parameters_atomically
kscottz@kscottz-ratnest:~$ ros2 action list
/turtle1/rotate_absolute
```

Let's begin digging into services. There seem to be quite a few services listed. Let's take a look at the `/spawn` service, which will create more turtles. ROS services and actions use messages similar to those used in topics to communicate. In fact, actions and services are built on top of messages. We can use the `service type` sub command to determine the message type used by a particular service. We can find specifics of the message by using the `interface show` command. Let's see this in practice with the `spawn` service:

```
$ ros2 service type /spawn
turtlesim/srv/Spawn
$ ros2 interface show turtlesim/srv/Spawn
float32 x
float32 y
float32 theta
string name # Optional. A unique name will be created and returned if this is
empty
---
string name
```

We can see from the output above that the spawn message takes three `float32` values for its position and orientation as well a `string` for its name. The `---` indicate the return value of the services. Unlike topics, services have a return value, which enables them to do things like perform computations and calculations.

Let's examine the help for calling a service by running `ros2 service call --help`:

```
$ ros2 service call --help
usage: ros2 service call [-h] [-r N] service_name service_type [values]
```

Call a service

positional arguments:

service_name	Name of the ROS service to call to (e.g. '/add_two_ints')
service_type	Type of the ROS service (e.g. 'std_srvs/srv/Empty')
values	Values to fill the service request with in YAML format (e.g. '{a: 1, b: 2}'), otherwise the service request will be published with default values

optional arguments:

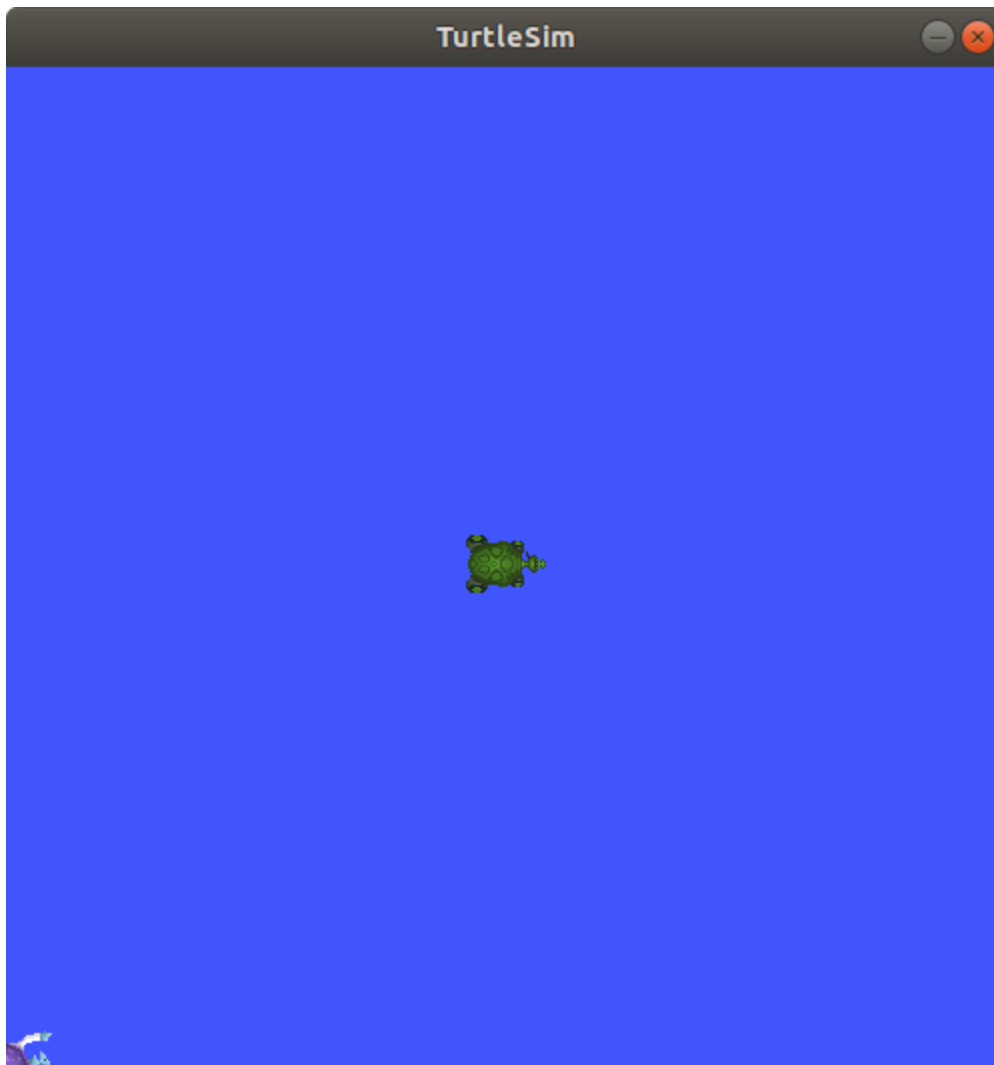
-h, --help	show this help message and exit
-r N, --rate N	Repeat the call at a specific rate in Hz

The syntax here is very similar to publishing to a topic, but instead of using a a topic name we use a service name. The service type is just like the topic type that we used in the past, but instead of using a message type we need a service type. Finally we give it a value in YAML format. A YAML string must be encased in single quotes. Let's try calling a service by creating a turtle named `Larry` at a position where all values are zero (use `TAB` complete).

```
$ ros2 service call /spawn turtlesim/srv/Spawn "{x: 0, y: 0, theta: 0.0, name: 'Larry'}"
requester: making request: turtlesim.srv.Spawn_Request(x=0.0, y=0.0, theta=0.0, name='Larry')

response:
turtlesim.srv.Spawn_Response(name='Larry')
```

If everything is working correctly you should now have a turtle named "Larry" in the lower left hand corner of the screen:



Try exploring the other services offered, or creating more turtles at different locations and moving them around.

Let's move on to actions. As mentioned previously, actions differ from services in a few ways and offer a number of advantages. Actions have the following advantages:

- Actions have a `goal`. That is to say you send them a goal, and they attempt to complete it.
- Actions can reject goal requests. This prevents them from becoming too busy.
- Actions are asynchronous and can perform tasks "while you wait."
- Actions will provide you with "updates" with information about their progress while you wait.
- Actions are preemptable, meaning you can cancel them if you change your mind.

Just like with services, we'll first figure out how to call the sole action in our ROS system by using the `action list`, `action show`, and `action info` commands. Recall, that when we called `ros2 action list` we got a single service. Now that we have Larry things have changed. Let's take a look:

```
$ ros2 action list
/Larry/rotate_absolute
/turtle1/rotate_absolute
```

Now there are two actions available, one for Larry and one for `turtle1`. Let's rotate `turtle1` to face Larry. First we'll call `action info` using `/turtle1/rotate_absolute` as the input and see what we get:

```
$ ros2 action info /turtle1/rotate_absolute
Action: /turtle1/rotate_absolute
Action clients: 0
Action servers: 1
  /turtlesim
```

Well, that tells us about the client and servers, but it really isn't helpful for our goal of moving Larry. Why don't we look at the `action send_goal` help and see if we can figure out how to use it:

```
$ ros2 action send_goal --help
usage: ros2 action send_goal [-h] [-f] action_name action_type goal
```

Send an action goal

positional arguments:

<code>action_name</code>	Name of the ROS action (e.g. <code>"/fibonacci"</code>)
<code>action_type</code>	Type of the ROS action (e.g. <code>"example_interfaces/action/Fibonacci"</code>)
<code>goal</code>	Goal request values in YAML format (e.g. <code>"{order: 10}"</code>)

optional arguments:

<code>-h, --help</code>	show this help message and exit
<code>-f, --feedback</code>	Echo feedback messages for the goal

This command needs an action name, an action type, and a goal in YAML. We know the action name, and we know how to write YAML, so all we need is to determine the action type. The best way to get the action type is the same way we published a message.

We see each of our turtles have one service called `rotate_absolute`. Let's dig into this action using the `info` sub command. This command has a `-t` flag to list the types of messages.

```
$ ros2 action info /turtle1/rotate_absolute -t
Action: /turtle1/rotate_absolute
Action clients: 0
Action servers: 1
  /turtlesim [turtlesim/action/RotateAbsolute]
```

The first line lists the action name. The second line gives the current number of clients for the action. The `Action servers` line gives the total number of action servers for this action. The last line gives the package and message type for the action.

We can see here that we need to know the action name, the type, and the values. Now the only problem is figuring out the format of the action type.

Let's understand the `RotateAbsolute` action message

The `ros2 interface show` command can be used to find the type of action message. Let's take a look:

```
$ ros2 interface show turtlesim/action/RotateAbsolute
# The desired heading in radians
float32 theta #< --- This section is the GOAL
---
# The angular displacement in radians to the starting position
float32 delta #< --- This section is the final result, different from the goal.
---
# The remaining rotation in radians
float32 remaining # < --- This is the current state.
```

What does this say about rotate absolute?

- There is a float input, `theta`, the desired heading. This first section is the actual goal.
- `delta` is the angle from the initial heading. This is the value returned when the action completes.
- `remaining` is the remaining radians to move. This is the value posted by the action while the action is being done.

With this information we can create our call to the action server. We'll use the `-f` flag to make this a bit clearer. Keep an eye on your turtle! It should move, slowly.

```
$ ros2 action send_goal -f /turtle1/rotate_absolute
turtlesim/action/RotateAbsolute {'theta: 1.70'}
Waiting for an action server to become available...
Sending goal:
  theta: 1.7

Feedback:
  remaining: 0.11599969863891602

Goal accepted with ID: 35c40e91590047099ae5bcc3c5151121

Feedback:
  remaining: 0.09999966621398926

Feedback:
  remaining: 0.06799960136413574

Feedback:
  remaining: 0.03599953651428223

Result:
  delta: -0.09600019454956055

Goal finished with status: SUCCEEDED
```

If everything worked correctly we should see our turtle has rotated.

ROS Parameters

In ROS, parameters are values that are shared between nodes in the system (if you are familiar with the [blackboard design pattern](#) in software engineering). Parameters are values that any node can query or write to. Another good analogy would be global constants in normal software programs. Parameters are best used to configure your robot. For example, if you were building an autonomous vehicle and wanted to cap the maximum velocity of the vehicle at 100 km/h, you could create a parameter called "MAX_SPEED" that is visible to all the nodes.

Let's take a look at the `param` command by running `ros2 param --help`.

```
$ ros2 param --help
Various param related sub-commands
```

Commands:

```
delete  Delete parameter
get      Get parameter
list     Output a list of available parameters
set      Set parameter
Call `ros2 param <command> -h` for more detailed usage.
```

At a high level, ROS 2's `param` command has sub commands to get and set a variable, along with a `list` functionality, and a `delete` command. As with most of the other commands we've worked through, it is instructive to look at `list` first. Let's see what the docs say about the `list` command and then see what happens when we call the sub command:

```
$ ros2 param list --help
usage: ros2 param list [-h] [--spin-time SPIN_TIME] [--include-hidden-nodes]
                        [--param-prefixes PARAM_PREFIXES [PARAM_PREFIXES ...]]
                        [node_name]
```

Output a list of available parameters

positional arguments:

```
node_name          Name of the ROS node
```

optional arguments:

```
-h, --help          show this help message and exit
--spin-time SPIN_TIME
                    Spin time in seconds to wait for discovery (only
                    applies when not using an already running daemon)
--include-hidden-nodes
                    Consider hidden nodes as well
--param-prefixes PARAM_PREFIXES [PARAM_PREFIXES ...]
                    Only list parameters with the provided prefixes
```

```
$ ros2 param list
/turtlesim:
background_b
background_g
background_r
use_sim_time
```

The only argument of note in this sub command is the `node_name` which allows you to narrow the scope of `param list` to only those parameters used by a particular node. In terms of parameters in the `turtlesim` node, we see that our call to `param list` gives us: three background color control params named `background_x` and a `use_sim_time` parameter. To learn all about the `param` command why don't we try to change these background color parameters using the CLI.

The first step in changing the background color is to see what the current color is. The `param get` sub command requires both a node name and a parameter name. In our list above we can see the node name as the top level element with the forward slash in front of it, namely `/turtlesim`. The syntax for `param get` is `ros2 param get <node_name> <param>`. Let's give it a whirl and see our current background color values.

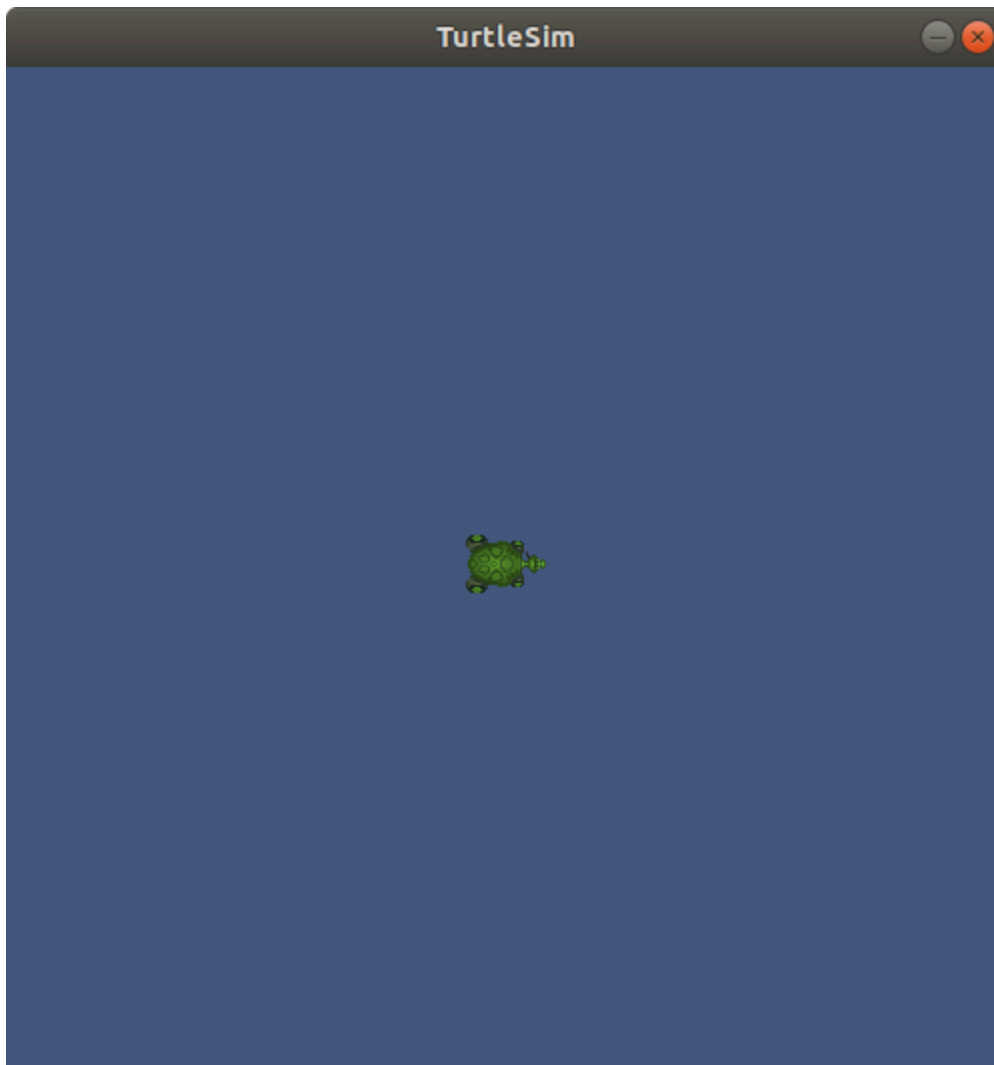
```
$ ros2 param get /turtlesim background_b
Integer value is: 255
$ ros2 param get /turtlesim background_g
Integer value is: 86
$ ros2 param get /turtlesim background_r
Integer value is: 69
```

On most computers color is represented as a triplet of <R,G,B> values. The color value of <69,86,255> corresponds to a periwinkle blue color. To change the color of the turtlesim we need to first set the parameter value and then reset the turtlesim to make it apply the color changes. We covered the basics of calling a service previously so we won't cover the steps of constructing a service call. Let's try setting our background color's blue component to 128.

```
$ ros2 param set turtlesim background_b 128
Set parameter successful
$ ros2 service call /reset std_srvs/srv/Empty
requester: making request: std_srvs.srv.Empty_Request()

response:
std_srvs.srv.Empty_Response()
```

If everything worked correctly your turtle should look like the screen below.



ROS Bags

ROS bags are ROS's tool for recording, and replaying data. ROS bags are like log files that let you store data along with messages. ROS systems can generate a lot of data, so when you bag data you must select which topics you want. Bags are a great tool for testing and debugging your application and a great tool for building robust unit tests.

Let's take a look at the root ROS Bag command by typing `ros2 bag --help` into the terminal. If you get an error you might need to install ROS Bag as it is often in a separate package. On Linux you can run `sudo apt install ros-eloquent-ros2bag` and it should automatically install the package for you.

```
$ ros2 bag -h
usage: ros2 bag [-h] Call `ros2 bag <command> -h` for more detailed usage. ...
```

Various rosbag related sub-commands

Commands:

```
info      ros2 bag info
play      ros2 bag play
record    ros2 bag record
```

As you can see there are three sub commands, `record`, `play`, and `info`. With these commands you can record a bag file, play/replay a file you've recorded, and find information about a bag file.

Let's try recording our first bag file. To do this we'll need three terminals all running ROS. The first terminal should already have our turtlesim running. If it isn't running you can restart it with `ros2 run turtlesim turtlesim_node`. Next you'll need to start the `draw_square` demo again to get the default turtle moving. To do this run `ros2 run turtlesim draw_square`. Now, in a third terminal we can bag some data by running the bag command. Let's first look at the record sub command by running `ros2 bag record -h`